

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA LINKED LIST



Nama :Muhammad Alfarisi

NIM :2025573010051

Kelas :TI /1C

PROGRAM STUDI TEKNIK INFORMATIKA JURUSAN
INFORMASI DAN KOMPUTER POLITEKNIK NEGERI
LHOKSEUMAWE

2026

DAFTAR ISI

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA LINKED LIST	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN	4
1. Latar Belakang.....	4
2. Identifikasi Masalah	4
2. Rumusan Masalah.....	4
BAB II LANDASAN TEORI	5
1. Pengertian Linked List	5
2. Pengertian Array.....	5
BAB III PROSES PRAKTIKUM	6
3.1. Alat Dan Bahan.....	6
3.2. Proses Praktikum Tugas-1.js.....	6
3.3. Proses pembuatan tugas-2.js	11
BAB IV PENUTUP	15
LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA	16
Linked Lish, Stack & Queue	16
BAB I PENDAHULUAN.....	17
1. Latar Belakang.....	17
2. Identifikasi Masalah	17
3. Rumusan Masalah.....	17
BAB II LANDASAN TEORI	18
1. Pengertian sctack dan queue.....	18
2. Struktur stack dan queue.....	18
3. Elemen stack dan queue	18
BAB III PROSES PRAKTIKUM	19
1. Alat Dan Bahan	19
2. Proses Praktikum	19

Linked List

BAB IV PENUTUP	28
4. kesimpulan	28

BAB I

PENDAHULUAN

1. Latar Belakang

sebagai alternatif dinamis untuk mengatasi kelemahan utama array, yaitu ukuran memori yang statis dan operasi penyisipan (insert) atau penghapusan (delete) yang lambat pada data dalam jumlah besar

2. Identifikasi Masalah

- Akses dan kinerja
- Masalah Manajemen Memori
- Kendala Logika Pemrograman (Bug)
- Tantangan Struktur (double dan circular Linked List)

2. Rumusan Masalah

- Apa yang dimaksud dengan linked list dan bagaimana konsep kerjanya dalam pemrograman?
- Apa saja jenis-jenis linked list (seperti Single, Double, dan Circular) dan karakteristiknya masing-masing?
- Bagaimana algoritma dan implementasi operasi dasar pada linked list (seperti insert, delete, search, dan display)?
- Apa saja kelebihan dan kekurangan linked list jika dibandingkan dengan struktur data lain seperti array?
- Dalam kasus atau aplikasi nyata seperti apa penggunaan linked list lebih efisien untuk diterapkan?

BAB II

LANDASAN TEORI

1. Pengertian Linked List

Linked list (atau senarai berantai) adalah struktur data linier yang terdiri dari kumpulan elemen data yang disebut node (simpul). Setiap node saling terhubung menggunakan pointer. Berbeda dengan array, linked list tidak menyimpan data secara berurutan di dalam memori, melainkan tersebar dan dihubungkan menggunakan referensi.

2. Pengertian Array

Array adalah data yang berada dalam pemrograman yang berfungsi untuk menyimpan sekumpulan data dengan tipe yang sama di dalam satu variabel.

3. Array VS Linked List

Array menyimpan elemen secara bersebelahan (contiguous) di memori. ini memungkinkan akses $O(1)$ via indeks, tapi insert/delete di tengah membutuhkan penggeseran elemen $\rightarrow O(n)$.

Linked List menyimpan elemen di mana saja di memori. Setiap elemen (disebut Node) menyimpan dua hal:

1. Data – nilai yang ingin disimpan
2. Next – referensi (pointer) ke node berikutnya

Ilustrasi Linked List: (HEAD) $\rightarrow$ [10| $\rightarrow$] $\rightarrow$ [20| $\rightarrow$] $\rightarrow$ [30| $\rightarrow$] $\rightarrow$ [40| null]

- node pertama disebut head
- node terakhir memiliki next = null (tandaanya akhir list)

Perbandingan Big O:

- Akses elemen ke-n : Array $O(1)$ VS Linked List $O(n)$
- Insert/Delete di HEAD: Array $O(n)$ VS Linked list $O(1)$
- Insert/Delete di Tail: Array $O(1)$ vs Linked list $O(n)$ [sigly]
- Insert/Delete di tengah: Array $O(n)$ VS Linked List $O(n)$ [tapi tanpa shift]
- Pencarian : Array $O(n)$ vs Linked List $O(n)$

Linked List unggul saat: sering insert/delete di awal, ukuran data sering berubah drastis. Array unggul saat: sering akses random via indeks, data ukuran tetap.

BAB III

PROSES PRAKTIKUM

3.1. Alat Dan Bahan

- Laptop
- Virtual Studio Code
- Git
- GitHub
- Node JS.

3.2. Proses Praktikum Tugas-1.js

Langkah 1 Buat Folder dan File

```
Mkdir tugas  
Touch tugas/tugas-1.js tugas/tugas-2.js
```

Langkah 2 Mengisi file tugas-1.js dengan kode berikut

```
//---mengimplementasikan class DoubleLikedList---  
//DoublyLikedlist = menyimpan node dalam dua arah (next  
dan prev)  
//head = nnode pertama  
//tail = node terakhir  
//size = jumlah node dalam list  
class Node {  
  constructor(data) {  
    this.data = data;  
    this.next = null;  
    this.prev = null;  
  }  
}
```

Linked List

```
}

class DoublyLinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
    this.size = 0;
  }

  // append: tambahkan node di akhir - O(1)
  append(data) {
    const newNode = new Node(data);
    if (!this.head) {
      this.head = newNode;
      this.tail = newNode;
    } else {
      this.tail.next = newNode;
      newNode.prev = this.tail;
      this.tail = newNode;
    }
    this.size++;
  }

  // prepend: tambahkan node di awal - O(1)
  prepend(data) {
    const newNode = new Node(data);
    if (!this.head) {
      this.head = newNode;
      this.tail = newNode;
    } else {
      newNode.next = this.head;
      this.head.prev = newNode;
      this.head = newNode;
    }
    this.size++;
  }

  // insertAt: sisipkan node di posisi tertentu - O(n)
  insertAt(data, index) {
    if (index < 0 || index > this.size) {
      console.log('Index di luar batas!');
      return;
    }
    if (index === 0) {
      this.prepend(data);
      return;
    }
  }
}
```

Linked List

```
    }
    if (index === this.size) {
      this.append(data);
      return;
    }

    const newNode = new Node(data);
    let current = this.head;
    for (let i = 0; i < index - 1; i++) {
      current = current.next;
    }
    const nextNode = current.next;
    current.next = newNode;
    newNode.prev = current;
    newNode.next = nextNode;
    nextNode.prev = newNode;
    this.size++;
  }

  // delete: hapus node pertama yang menemukan data -
  O(n)
  delete(data) {
    if (!this.head) return false;

    let current = this.head;
    while (current && current.data !== data) {
      current = current.next;
    }

    if (!current) return false;

    if (current === this.head) {
      this.head = this.head.next;
      if (this.head) this.head.prev = null;
      else this.tail = null;
    } else if (current === this.tail) {
      this.tail = this.tail.prev;
      this.tail.next = null;
    } else {
      current.prev.next = current.next;
      current.next.prev = current.prev;
    }

    this.size--;
    return true;
  }
```


Linked List

```
// reverse: membalik urutan list - O(n)
reverse() {
  let current = this.head;
  let temp = null;

  while (current) {
    temp = current.prev;
    current.prev = current.next;
    current.next = temp;
    current = current.prev;
  }

  if (temp) {
    this.tail = this.head;
    this.head = temp.prev;
  }
}

// print: menampilkan dari depan dan dari belakang -
O(n)
print() {
  let forward = '';
  let current = this.head;
  while (current) {
    forward += current.next ? `[${current.data}]`
    <-> ` : `[${current.data}]`;
    current = current.next;
  }
  console.log('Forward:', forward);

  let backward = '';
  current = this.tail;
  while (current) {
    backward += current.prev ?
    `[${current.data}] <-> ` : `[${current.data}]`;
    current = current.prev;
  }
  console.log('Backward:', backward);
  console.log(`size: ${this.size}`);
}

}

//--- demonstrasi penggunaan DoublyLinkedList ---
if (require.main === module) {
  const dll = new DoublyLinkedList();
```

Linked List

```
    console.log('=== append ===');  
    dll.append(10);  
    dll.append(20);  
    dll.append(30);  
    dll.print();  
  
    console.log('\n=== prepend ===');  
    dll.prepend(5);  
    dll.print();  
  
    console.log('\n=== insertAt(25, 3) ===');  
    dll.insertAt(25, 3);  
    dll.print();  
  
    console.log('\n=== delete(20) ===');  
    dll.delete(20);  
    dll.print();  
  
    console.log('\n=== reverse ===');  
    dll.reverse();  
    dll.print();  
  
}
```

Hasil	Run terminal untuk file tugas-1.js
-------	------------------------------------

Linked List, stack dan queue

Cd Praktikum-6/tugas/tugas-1.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-6/tu
$ node tugas-1.js
=== append ===
Forward: [10] <-> [20] <-> [30]
Backward: [30] <-> [20] <-> [10]
size: 3

=== prepend ===
Forward: [5] <-> [10] <-> [20] <-> [30]
Backward: [30] <-> [20] <-> [10] <-> [5]
size: 4

=== insertAt(25, 3) ===
Forward: [5] <-> [10] <-> [20] <-> [25] <-> [30]
Backward: [30] <-> [25] <-> [20] <-> [10] <-> [5]
size: 5

=== delete(20) ===
Forward: [5] <-> [10] <-> [25] <-> [30]
Backward: [30] <-> [25] <-> [10] <-> [5]
size: 4

=== reverse ===
Forward: [30] <-> [25] <-> [10] <-> [5]
Backward: [5] <-> [10] <-> [25] <-> [30]
size: 4
```

3.3. Proses pembuatan tugas-2.js

Langkah 1 Masuk ke file tugas-2.js dan isi code seperti dibawah

```
//---Membuat node dan fungsi-fungsi untuk LinkedList---

class node {
  constructor(data) {
    this.data = data;
    this.next = null;
  }
}

// membuat linkedlist dari array, dan kembalikan head dari linkedlist
function fromArray(arr) {
  let head = null, tail = null;
  for (const v of arr) {
    const newNode = new node(v);
```

Linked List, stack dan queue

```
        if (!head) head = tail = newNode;
        else { tail.next = newNode; tail = newNode;}
    }
    return head;
}

// membuat konversi linked list ke array (untuk verifikasi)
function toArray(head) {
    const out = [];
    let cur = head;
    while (cur) { out.push(cur.data); cur = cur.next; }
    return out;
}

// fungsi untuk memeriksa apakah linked list merupakan palindrome
function palindromLL(head) {
    const arr = [];
    let cur = head;
    while (cur) {arr.push(cur.data); cur = cur.next; }
    let i = 0, j = arr.length - 1;
    while (i < j) {
        if (arr[i] !== arr[j]) return false;
        i++; j--;
    }
    return true;
}

// hapus n dari akhir (head, n)
function hapusNDariAkhir(head, n) {
    const dummy = new node(null);
    dummy.next = head;
    let fast = dummy, slow = dummy;
    for (let i = 0; i < n; i++){
        if (!fast.next) return head; // jika n > panjang = tidak akan
        berubah
        fast = fast.next;
    }

    // maju sampai fast mencapai akhir
    while (fast.next) {
        fast = fast.next;
        slow = slow.next;
    }

    // slow.next = node yang akan dihapus
    slow.next = slow.next ? slow.next.next : null;
}
```

Linked List, stack dan queue

```
    return dummy.next;
}

// kompleksitas: O(n) waktu, O(1) ruang.
function tengahLinkedList(head) {
    if (!head) return null;
    let slow = head, fast = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow; // jika nilai genap, maka slow akan berada pada node
    tengah kedua
}

// contoh kasus per fungsi
function test() {
    //palindromLL
    console.log(palindromLL(fromArray([10, 20, 30, 20, 10])) ===
true);
    console.log(palindromLL(fromArray([10, 20, 30, 40])) === false);
    console.log(palindromLL(fromArray([10])) === true);

    //menghapusNdariakhir
    console.log(toArray(hapusNDariAkhir(fromArray([10, 20, 30, 40,
50]), 2))); // [10, 20, 30, 50]
    console.log(toArray(hapusNDariAkhir(fromArray([10,20]), 2)));
    // [20] (menghapus head)
    console.log(toArray(hapusNDariAkhir(fromArray([10,20,30]), 4)));
    // n > len -> [10,20,30] tidak berubah

    //tengahLinkedList
    console.log(tengahLinkedList(fromArray([1,2,3])).data === 2);
    console.log(tengahLinkedList(fromArray([1,2,3,4])).data === 3);
    //node tengah kedua
    console.log(tengahLinkedList(fromArray([1])).data === 1);
}

if (require.main === module) test();
```

Hasil Run terminal untuk file tugas-2.js

Node tugas-2.js

Linked List, stack dan queue

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-6/tugas (main)
$ node tugas-2.js
true
true
true
[ 10, 20, 30, 50 ]
[ 20 ]
[ 10, 20, 30 ]
true
true
true
```

BAB IV PENUTUP

Linked List adalah struktur data linear dinamis yang terdiri dari kumpulan node. Setiap node menyimpan data dan pointer (referensi) ke node selanjutnya. Hal ini menjadikannya sangat fleksibel dalam mengalokasikan memori dibandingkan array biasa.

LAPORAN PRAKTIKUM ALGORITMA DAN SRUKTUR DATA Linked Lish, Stack & Queue



Nama :Muhammad Alfarisi

NIM :2025573010051

Kelas :TI /1C

PROGRAM STUDI TEKNIK INFORMATIKA JURUSAN
INFORMASI DAN KOMPUTER POLITEKNIK NEGERI
LHOKSEUMAWE

2026

BAB I

PENDAHULUAN

1. Latar Belakang

Latar belakang ketiga struktur data ini berakar pada kebutuhan komputasi untuk menyimpan, mengelola, dan memanipulasi memori secara efisien. Masing-masing diciptakan untuk menyelesaikan batasan memori statis (seperti Array) dan menyesuaikan dengan perilaku alur data di dunia nyata, baik itu manajemen memori dinamis, tumpukan tugas, maupun sistem antrian.

2. Identifikasi Masalah

- Kelemahan point/Refere
- Masalah pada Queue
- Mengatasi Masalah dengan Pendekatan Dinamis

3. Rumusan Masalah

- 1) Mengimplementasikan Stack dan Queue menggunakan Linked List (bukan array) agar semua operasi utama $O(1)$?
- 2) Memahami kapan Stack lebih tepat vs Queue untuk memecahkan masalah tertentu?
- 3) Menggunakan Queue untuk simulasi BFS (Breadth-First Search) sederhana?
- 4) Mengimplementasikan Deque (Double-Ended Queue) sebagai generalisasi Stack+Queue?

BAB II

LANDASAN TEORI

1. Pengertian stack dan queue

Dalam struktur data pemrograman, Stack dan Queue adalah dua metode dasar yang digunakan untuk menyimpan dan mengatur data. Stack beroperasi dengan prinsip Last In, First Out (LIFO)—data terakhir yang dimasukkan akan menjadi yang pertama keluar—sedangkan Queue beroperasi dengan prinsip First In, First Out (FIFO)—data pertama yang masuk akan menjadi yang pertama keluar

2. Struktur stack dan queue

- struktur data stack (LIFO)
- struktur data queue (FIFO)

3. Elemen stack dan queue

- stack: last in first out (LIFO)
- queue: first in, first-out (FIFO)

BAB III

PROSES PRAKTIKUM

1. Alat Dan Bahan

- Laptop
- Virtual Studio Code
- Git
- GitHub
- Node JS.

2. Proses Praktikum

Langkah 1 Membuat folder Praktikum-7 di dalam repository.

```
Mkdir praktikum-7  
Cd praktikum-7
```

Langkah 2 Membuat folder Untuk Tugas

```
Mkdir tugas
```

Langkah 3 Membuat file untuk isi folder tugas

```
Cd tugas touch tugas-1.js tugas-2.js
```

Langkah 4 Isi file tugas-1.js dengan code seperti code dibawah

```
class Pasien {
  constructor(id, nama, prioritas, waktuDaftar) {
    this.id = id
    this.nama = nama
    this.prioritas = prioritas
    this.waktuDaftar = waktuDaftar
  }

  toString() {
    return `ID:  ${this.id},  Nama:  ${this.nama},
Prioritas:  ${this.prioritas},  Waktu    Daftar:
${this.waktuDaftar.toLocaleString()}`
  }
}

class AntrianRS {
  constructor() {
    this.antrianDarurat = []
    this.antrianBiasa = []
  }

  daftar(pasien) {
```

linked list, Stack dan queue

```
    if (pasien.prioritas === 'darurat') {
        this.antrianDarurat.push(pasien)
    } else {
        this.antrianBiasa.push(pasien)
    }
}

layani() {
    if (this.antrianDarurat.length > 0) {
        const pasien = this.antrianDarurat.shift()
        console.log(`Sedang dilayani (darurat):
${pasien.toString()}`)
        return pasien
    }

    if (this.antrianBiasa.length > 0) {
        const pasien = this.antrianBiasa.shift()
        console.log(`Sedang dilayani (biasa):
${pasien.toString()}`)
        return pasien
    }

    console.log('Tidak ada pasien yang harus dilayani.')
    return null
}

tampilkanAntrian() {
```

[linked list, Stack dan queue](#)

```
    console.log('=== Status Antrian ===')
    console.log('Antrian Darurat:')
    if (this.antrianDarurat.length === 0) {
        console.log('    (kosong)')
    } else {
        this.antrianDarurat.forEach((pasien, index) => {
            console.log(`        ${index}    +    1}.
${pasien.toString()}`)
        })
    }

    console.log('Antrian Biasa:')
    if (this.antrianBiasa.length === 0) {
        console.log('    (kosong)')
    } else {
        this.antrianBiasa.forEach((pasien, index) => {
            console.log(`        ${index}    +    1}.
${pasien.toString()}`)
        })
    }
    console.log('=====')
}

function buatSimulasi() {
    const rs = new AntrianRS()
    const namaPasien = [
```

linked list, Stack dan queue

```
'Andi',  
'Budi',  
'Citra',  
'Dewi',  
'Eka',  
'Fajar',  
'Gina',  
'Hadi',  
'Ika',  
'Joko'  
]  
  
namaPasien.forEach((nama, index) => {  
    const prioritas = Math.random() < 0.5 ? 'darurat' :  
'biasa'  
  
    const waktuDaftar = new Date(Date.now() + index *  
1000)  
  
    const pasien = new Pasien(index + 1, nama, prioritas,  
waktuDaftar)  
    rs.daftar(pasien)  
    console.log(`Daftar: ${pasien.toString()}`)  
})  
  
console.log(`\nMulai simulasi...\n`)  
rs.tampilkanAntrian()  
  
while (rs.antrianDarurat.length > 0 ||  
rs.antrianBiasa.length > 0) {
```

linked list, Stack dan queue

```

    rs.layani()
  }

  console.log('\nSemua pasien telah dilayani.')
}

buatSimulasi()

```

Langkah 5 Simpan dan jalankan pada terimanal

Node tugas-1.js

```

ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-7/tugas (main)
$ node tugas-1.js
Daftar: ID: 1, Nama: Andi, Prioritas: biasa, Waktu
Daftar: 6/9/2026, 9:09:34 PM
Daftar: ID: 2, Nama: Budi, Prioritas: darurat, Wakt
u Daftar: 6/9/2026, 9:09:35 PM
Daftar: ID: 3, Nama: Citra, Prioritas: darurat, Wak
tu Daftar: 6/9/2026, 9:09:36 PM
Daftar: ID: 4, Nama: Dewi, Prioritas: biasa, Waktu
Daftar: 6/9/2026, 9:09:37 PM
Daftar: ID: 5, Nama: Eka, Prioritas: biasa, Waktu D
aftar: 6/9/2026, 9:09:38 PM
Daftar: ID: 6, Nama: Fajar, Prioritas: darurat, Wak
tu Daftar: 6/9/2026, 9:09:39 PM
Daftar: ID: 7, Nama: Gina, Prioritas: darurat, Wakt
u Daftar: 6/9/2026, 9:09:40 PM
Daftar: ID: 8, Nama: Hadi, Prioritas: darurat, Wakt
u Daftar: 6/9/2026, 9:09:41 PM
Daftar: ID: 9, Nama: Ika, Prioritas: biasa, Waktu D
aftar: 6/9/2026, 9:09:42 PM
Daftar: ID: 10, Nama: Joko, Prioritas: biasa, Waktu
Daftar: 6/9/2026, 9:09:43 PM

Mulai simulasi...

=== Status Antrian ===
Antrian Darurat:
  1. ID: 2, Nama: Budi, Prioritas: darurat, Waktu D
aftar: 6/9/2026, 9:09:35 PM
  2. ID: 3, Nama: Citra, Prioritas: darurat, Waktu
Daftar: 6/9/2026, 9:09:36 PM
  3. ID: 6, Nama: Fajar, Prioritas: darurat, Waktu
Daftar: 6/9/2026, 9:09:39 PM
  4. ID: 7, Nama: Gina, Prioritas: darurat, Waktu D
aftar: 6/9/2026, 9:09:40 PM

```


Langkah 6 Masuk ke file tugas-2.js dan isi code seperti dibawah

```
class MinStack {
  constructor() {
    this.tumpukan = []
    this.tumpukanMin = []
  }

  push(nilai) {
    // Kompleksitas waktu: O(1)
    this.tumpukan.push(nilai)

    if (this.tumpukanMin.length === 0 || nilai <=
this.tumpukanMin[this.tumpukanMin.length - 1]) {
      this.tumpukanMin.push(nilai)
    }
  }

  pop() {
    // Kompleksitas waktu: O(1)
    if (this.tumpukan.length === 0) {
      return null
    }

    const nilai = this.tumpukan.pop()
    if (nilai === this.tumpukanMin[this.tumpukanMin.length
```

[linked list, Stack dan queue](#)

```
- 1]) {
    this.tumpukanMin.pop()
}
return nilai
}

top() {
    // Kompleksitas waktu: O(1)
    return this.tumpukan.length === 0 ? null :
this.tumpukan[this.tumpukan.length - 1]
}

getMin() {
    // Kompleksitas waktu: O(1)
    return this.tumpukanMin.length === 0 ? null :
this.tumpukanMin[this.tumpukanMin.length - 1]
}
}

const minStack = new MinStack()
minStack.push(5)
minStack.push(3)
minStack.push(7)
minStack.push(2)
console.log('getMin() =>', minStack.getMin()) // 2

minStack.pop()
```

linked list, Stack dan queue

```
console.log('getMin() =>', minStack.getMin()) // 3

minStack.pop()

console.log('getMin() =>', minStack.getMin()) // 3
```

Hasil	Run tterminal untuk file tugas-2.js
-------	-------------------------------------

```
ASUS@LAPTOP-68I2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-7/tugas (main)
$ node tugas-2.js
getMin() => 2
getMin() => 3
getMin() => 3
```

BAB IV PENUTUP

4. kesimpulan

Kesimpulan utama antara Stack dan Queue terletak pada metode manajemen dan akses datanya. Stack menggunakan prinsip LIFO (Last In, First Out), sedangkan Queue menggunakan prinsip FIFO (First In, First Out). Keduanya adalah struktur data linear yang sangat penting dalam pemrograman.

.